Data clustering

Lecturer: Assoc.Prof. Nguyễn Phương Thái

VNU University of Engineering and Technology
Slide: *from Assoc.Prof. Phan Xuân Hiếu, Updated: September 05, 2023*

Outline

1. Data clustering concepts
2. Data understanding before clustering
3. Hierarchical clustering
4. Partitioning clustering
5. Distribution–based clustering
6. Density–based clustering
7. Clustering validation and evaluation
8. References and Summary

Outline

1. Data clustering concepts
2. Data understanding before clustering
3. Hierarchical clustering
4. Partitioning clustering
5. Distribution–based clustering
6. Density–based clustering
7. Clustering validation and evaluation
8. References and Summary

What is data clustering?

Definition from Data Mining: Concepts and Techniques, J. Han et al. [1]

Cluster analysis *or simply* ***clustering*** *is the process of partitioning a set of data objects (or observations) into subsets. Each subset is a* ***cluster****, such that objects in a cluster are similar to one another, yet dissimilar to objects in other clusters.*

Definition from Mining of Massive Datasets, J. Leskovec et al. [3]

Clustering is the process of examining a collection of ***points****, and grouping the points into* ***clusters*** *according to some distance measure. The goal is that points in the same cluster have a small distance from one another, while points in different clusters are at a large distance from one another.*

Definition from Wikipedia

Cluster analysis *or* ***clustering*** *is the task of grouping a set of objects in such a way that objects in the same group (called a* ***cluster****) are more similar (in some sense) to each other than to those in other groups (clusters).*

Data clustering (cont'd)

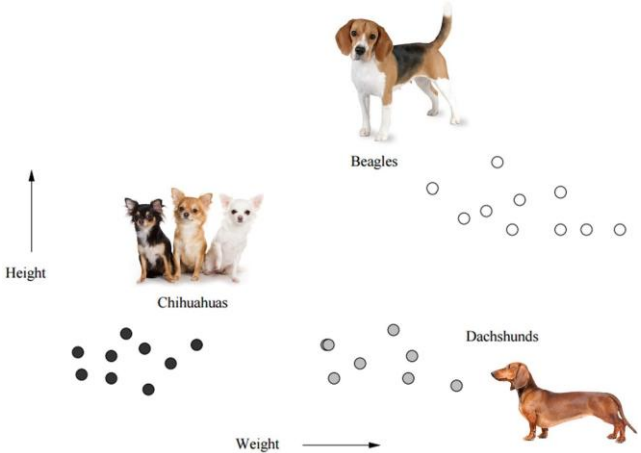
- Data clustering is also called *unsupervised learning* or *unsupervised classification*.
- Classification (supervised learning) is *learning by examples* whereas clustering is *learning by observation*.
- Two main types of clustering:
  - Hard clustering: each data point belongs to only one cluster.
  - Soft clustering: each data point can belong to one or more clusters.
- Some characteristics:
  - The number of clusters of a dataset is normally unknown, or not really clear.
  - There are several clustering approaches, each has several clustering techniques.
  - Different clustering approaches/techniques may give different results.

Data clustering problem



- Let $X = (X_1, X_2, \ldots, X_d)$ be a d–dimensional space, where each attribute/variable X_j is numeric or categorical.
- Let $D = \{x_1, x_2, \ldots, x_n\}$ be a data sample or dataset consisting of n data points (a.k.a data instances, observations, examples, or tuples) $x_i = (x_1, x_2, \ldots, x_d) \in X$.
- Data clustering is to use a clustering technique or algorithm A to assign data points in D into their most likely clusters. The clustering results are a set of k clusters $C = \{C_1, C_2, \ldots, C_k\}$. Data points in the same cluster are similar to each other in some sense and far from the data points in other clusters.

Example of data points in 2–dimensional space [3]



Observations about the clustering process and the results

- The number of clusters k is specified in two ways: (1) k is an **input parameter** of the clustering algorithm, and (2) k can be **determined automatically** by the algorithm.
- Normally, each data point belongs only one cluster (i.e., hard clustering).
- If data points belong to more than one clusters (soft clustering), the **membership** of x_i in a cluster C_j is characterized by a **weight** w_{ij} (e.g., in range $[0, 1]$).
- Not all data points in D are assigned into clusters. There may be several data points that are **outliers** or **noise** and they are excluded from the clusters.
- The clustering results depend on clustering algorithms. Some algorithms is for hard clustering, some for soft clustering, some can deal with outliers and noise.
- The cluster assignment for data points is performed automatically by clustering algorithms. Hence, clustering is useful in that it can lead to the **discovery** of previously **unknown** groups within the data.

Requirements for data clustering

- **Scalability**: clustering algorithms should be able to work with small, medium, and large datasets with a consistent performance.
- **Ability to deal with different types of attributes**: clustering algorithms can work with different data types like binary, nominal (categorical), ordinal, numeric, or mixtures of those data types.
- **Discovery of clusters with arbitrary shape**: algorithms based on such distance measures tend to find **spherical** clusters with similar size and density. However, a cluster could be of any shape. It is important to develop algorithms that can detect clusters of arbitrary shape.
- **Requirements for domain knowledge to determine input parameters**: clustering should be as automatic as possible, voiding (biased) domain knowledge.
- **Ability to deal with noisy data**: most real–world data sets contain outliers and/or missing, unknown, or erroneous data. Clustering algorithms can be sensitive to such noise and may produce poor–quality clusters. Therefore, we need clustering methods that are robust to noise.

Requirements for data clustering (cont'd)

- **Incremental clustering and insensitivity to input order**: in many applications, incremental updates (representing newer data) may arrive at any time. It is better if clustering algorithms can handle future data points in an incremental manner.
- **Capability of clustering high–dimensionality data**: a data set can contain numerous dimensions or attributes. Finding clusters of data objects in a high-dimensional space is challenging, especially considering that such data can be very sparse and highly skewed.
- **Constraint–based clustering**: real–world applications may need to perform clustering under various kinds of constraints, e.g., two particular data points cannot be in the same cluster or vice versa. Constraint integration into clustering algorithms is important in some application domains.
- **Interpretability and usability**: users want clustering results to be interpretable, comprehensible, and usable. That is, clustering may need to be tied in with specific semantic interpretations and applications.

Clustering approaches

- **Hierarchical methods**: also called connectivity methods
  - Create a hierarchical decomposition of data, i.e., a tree of clusters (**dendrogram**).
  - Hierarchical clustering can be **agglomerative** (bottom–up) or **divisive** (top–down).
  - Use various similarity measures split or merge clusters.
  - This approach is **hard clustering**. The resulting clusters are in **spherical shape**.
- **Partitioning methods**: also called centroid methods
  - Data points are partitioned into k exclusive clusters (k is an input parameter).
  - Both **centroid–based** and **distance–based**.
  - Well–known techniques: k–means, k–medoids, k–medians, etc.
  - This approach is also **hard clustering**.
  - Suitable for finding **spherical–shaped** clusters in small– to medium–size databases.

Clustering approaches (cont'd)

- **Distribution–based methods**: also called **probabilistic** models
  - Assuming data points are from a **mixture** of distributions, e.g., normal distributions.
  - Well–known methods: Gaussian mixture models (**GMMs**) with expectation maximization (**EM**) algorithm.
  - This is **soft clustering**. The clusters can overlap and have **elliptical** shapes.
  - For clusters in **arbitrary shapes**, distribution methods may **fail** because the distribution **assumption** is normally **wrong**.
- **Density–based methods**:
  - Idea: continue to grow a cluster as long as the **density** (number of objects or data points) in the **neighborhood** exceeds some **threshold**.
  - This approach is suitable for clusters of **arbitrary shapes**.
  - This approach can also deal with **noise** and **outliers**.
  - Most common algorithms are **DBSCAN** and **OPTICS**.

Clustering approaches (cont'd)

- **Grid–based methods**:
  - Quantize the object space into a finite number of **cells** that form a **grid structure**.
  - All the clustering operations are performed on the grid structure.
  - Advantage: **fast** processing time, depending on the number of cells.
  - Efficient for **spatial** data clustering, can be combined with density–based method, etc.
- **Other approaches**:
  - **Graph–based methods**:
    finding clusters based on **dense sub–graph** mining like **cliques** or **quasi–cliques**.
  - **Subspace models**:
    clusters are modeled with both cluster members and relevant attributes.
  - **Neural models**:
    the most well known unsupervised neural network is the **self–organizing map**.

Clustering approaches (cont'd)

Method	General Characteristics
Partitioning methods	– Find mutually exclusive clusters of spherical shape – Distance-based – May use mean or medoid (etc.) to represent cluster center – Effective for small- to medium-size data sets
Hierarchical methods	– Clustering is a hierarchical decomposition (i.e., multiple levels) – Cannot correct erroneous merges or splits – May incorporate other techniques like microclustering or consider object "linkages"
Density-based methods	– Can find arbitrarily shaped clusters – Clusters are dense regions of objects in space that are separated by low-density regions – Cluster density: Each point must have a minimum number of points within its "neighborhood" – May filter out outliers
Grid-based methods	– Use a multiresolution grid data structure – Fast processing time (typically independent of the number of data objects, yet dependent on grid size)

Challenges in data clustering

- Clustering with a **high volume** of data.
- Clustering in **high–dimensional** space.
- Clustering with **low–quality** data (e.g., noisy and missing values).
- Clustering with **complex cluster structures** (shape, density, overlapping, etc.).
- Identifying **right values for parameters** that can reflects the nature of data (e.g., the right number of clusters, the right density, etc.)
- **Validation** and **assessment** of clustering results.

Clustering in high–dimensional space: the curse of dimensionality

In very high–dimensional space, **almost pairs** of data points are **far from each other**. Given two data points $\mathbf{x}$ and $\mathbf{y}$ in $\mathcal{R}^d$. The Euclidean distance $r(\mathbf{x}, \mathbf{y})$ is:

$$r(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2}$$

Suppose all data points are in the **unit cube** (i.e., all dimension in $[0, 1]$). The lower bound of $r(\mathbf{x}, \mathbf{y})$ is 1 (likely that there is at least one dimension where $\mathbf{x}$ and $\mathbf{y}$ very far from each other); and the **upper bound** is $\sqrt{d}$. In fact, most of pairs of data points have a distance of the average: $(1 + \sqrt{d})/2$. For d **very large**, the **distance** between data points is **large**.

Clustering in high–dimensional space: the curse of dimensionality (2)

- In a very high–dimensional space, **two arbitrary vectors** are nearly **orthogonal**. Consider the cosine similarity:

$$\cos(\mathbf{x}, \mathbf{y}) = \frac{\sum_{i=1}^{d} x_{yi}}{\sqrt{\sum_{i=1}^{d} x_i^2}\sqrt{\sum_{i=1}^{d} y_i^2}} \quad (2)$$

- When d is very large, the **numerator** is **much smaller** than the **denominator**, and the cosine between the two vectors is very **close to zero**.

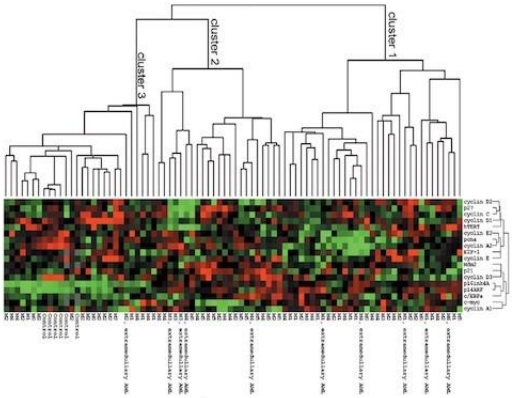
- If most of pairs of data points are orthogonal, it is **very hard** to perform clustering. The **clustering results** are normally **very bad**.

- One of the solution is **dimensionality reduction** with popular techniques like PCA, SVD, or topic analysis or word embeddings (for text data).

Applications of data clustering



[nguồn: buffy.eecs.berkeley.edu]



[nguồn: www.nature.com]

Applications of data clustering (cont'd)

- Customer segmentation (telco, retail, marketing, finance and banking, etc.)
- Text clustering (news, email, customer care data, tag suggestion, etc.)
- Image processing, object segmentation, etc.
- Biological data clustering (patients, health records, gene, etc.)
- Finding similar users and sub–communities (graph, social networks, etc.)
- Buyer and product clustering (retail, recommender systems, etc.)
- Identifying fraudulent or criminal activities, etc.
- Clustering can be a preprocessing step for further data analysis and mining.
- Any data mining tasks that require to group data points into similar clusters. The applications can be found everywhere in data analysis and mining.

Outline

1 Data clustering concepts

2 Data understanding before clustering

3 Hierarchical clustering

4 Partitioning clustering

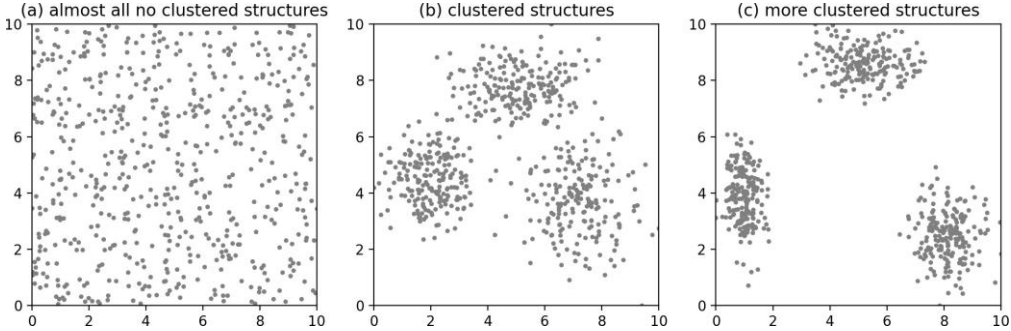
5 Distribution–based clustering

6 Density–based clustering

7 Clustering validation and evaluation

8 References and Summary

Understanding of data distribution



- Do the data have cluster structures? Is the data clusterable (clusterability)?
- How to assess the data distribution mathematically and automatically?

Clustering tendency identification methods

- Spatial histogram (cell–based histogram)
- Cell–based entropy
- Distance distribution
- Hopkins statistic

Validating cluster tendency with spatial histogram

- A simple approach is to contrast the d–dimensional spatial histogram of the dataset $\mathtt{D}$ with the histogram from samples generated randomly in the same data space.
- Let $X_1, X_2, \ldots, X_d$ denote the d dimensions. Given b, the number of bins for each dimension, we divide each dimension X_j into b equi–width bins, and simply count how many points lie in each of the b^d d–dimensional cells.
- From these d–histograms, we can obtain the **empirical joint probability mass function** (EPMF) for the dataset $\mathtt{D}$, which is an approximation of the unknown joint probability density function. The EPMF is given as

$$f(\mathbf{i}) = P(\mathbf{x}_j \in \text{cell } \mathbf{i}) = \frac{|\{\mathbf{x}_j \in \text{cell } \mathbf{i}\}|}{n} \tag{3}$$

where $\mathrm{i} = (i_1, i_2, \ldots, i_d)$ denotes a cell index, with i_j denoting the bin index along dimension X_j; $n = |\mathtt{D}|$.

Validating cluster tendency with spatial histogram (cont'd)

- Next, we generate t random samples, each comprising n points within the same d–dimensional space as the input dataset $\mathtt{D}$. That is, for each dimension X_j, we compute its range $[min(X_j), max(X_j)]$, and generate values **uniformly** at random with the given range. Let R_j denote the j–th such random sample.
- Compute the corresponding EPMF $g_j(\mathrm{i})$ for each R_j, $1 \leq j \leq t$.
- Compute how much the distribution f differs from g_j (for $j = 1..t$) using the Kullback–Leibler (KL) divergence from f to g_j, defined as:

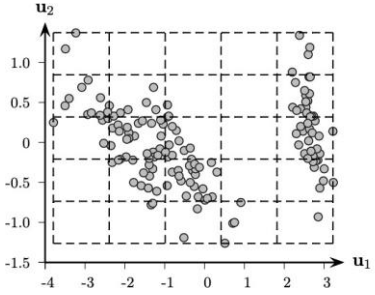
$$KL(f|g_j) = \sum_{\mathbf{i}} f(\mathbf{i}) \log \left(\frac{f(\mathbf{i})}{g_j(\mathbf{i})} \right) \tag{4}$$

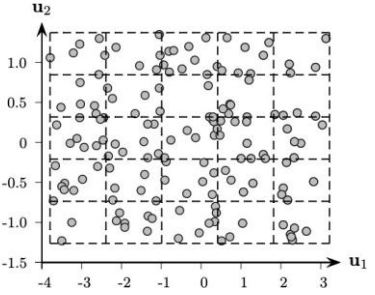
- The KL divergence is zero only when f and g_j are the same distributions. Using these divergence values, we can compute how much the dataset $\mathtt{D}$ differs from a random dataset.
- Compute the expectation and the variance of $KL(f|g_j)$ (for $j = 1..t$).

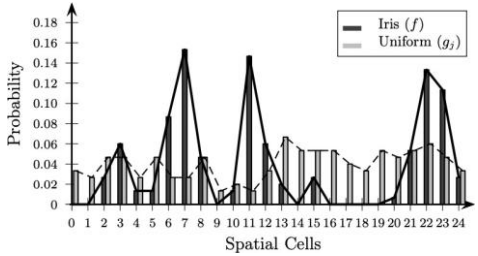
Example of spatial histogram [4]

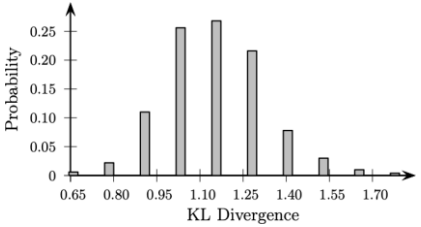
- The main limitation of this approach is that as dimensionality increases, the number of cells (b^d) increases exponentially, and with a fixed sample size n, most of the cells will be empty, or will have only one point, making it hard to estimate the divergence. The method is also sensitive to the choice of parameter b.
- The example in the next slide shows the empirical joint probability mass function for the Iris principal components dataset that has $n = 150$ points in $d = 2$ dimensions.
- It also shows the EPMF for one of the datasets generated uniformly at random in the same data space. Both EPMFs were computed using $b = 5$ bins in each dimension, for a total of 25 spatial cells.
- With $t = 500$, and computed the KL divergence from f to g_j for each $1 \leq j \leq t$ (using logarithm with base 2).
- The mean KL value was $\mu_{KL} = \mathbf{1.17}$, with a standard deviation of $\sigma_{KL} = \mathbf{0.18}$, indicating that the Iris data is indeed far from the randomly generated data, and thus is clusterable.

Example of spatial histogram [4] (cont'd)

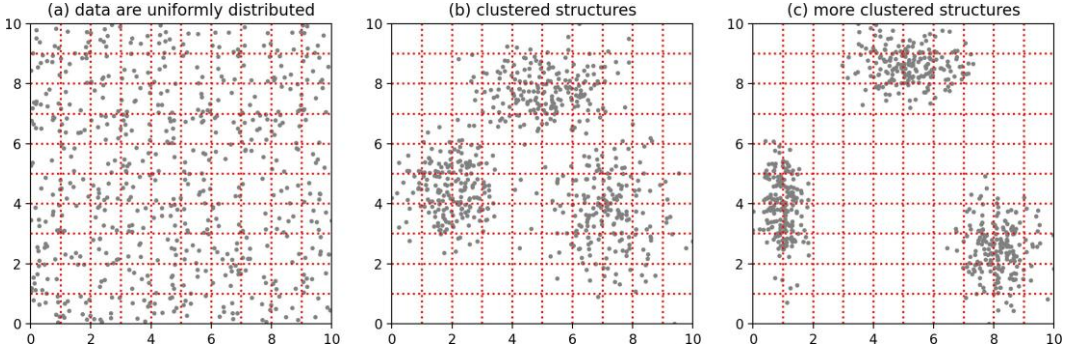








Validating cluster tendency with cell–based entropy



- The data space is divided into a grid of $k \times k$ cells. For instance, $k = 10$, and then the total number of cells is $m = 100$ cells.
- Counting the number of data points in each cells for three cases (a), (b), and (c).

Validating cluster tendency with cell–based entropy (cont'd)

```
[ 8 12  7 11  5  3  8  9  9  7]
[11  2  4  3  7  5  6  4  6  7]
[ 7  4  8  7  8  3  9  4  6  7]
[ 6  6  4  7  7  9  8  7  6  6]
[ 7  6  6  8  4  4  9  6  5  8]
[ 5  5  6  7  8  8  5  7  4  4]
[ 7  9  7  6  4  3  7  2  7  7]
[ 6  3  6  3  2  4 11  4  4  5]
[ 5  4  3  5  5  5  7  8  7  4]
[ 4  7  6  6  6  4  2  5  8  7]
```

```
[ 0  0  1  4 10  6  0  0  0  0]
[ 0  0  2 23 27 22  4  0  0  0]
[ 0  0  5 22 25 25  5  2  4  0]
[ 0  0  0  7  9  3  6 12 10  1]
[ 0  0  1  1  3  1  8 29 19  2]
[ 0  1  3  5  6  2  5 34 14  3]
[ 1  3 14 23 13 11  7 19 14  2]
[ 0  7 12 23 21  8  2  3  5  2]
[ 0  5  7  5  6  3  2  0  0  0]
[ 1  0  3  3  2  1  0  0  0  0]
```

```
[ 0  2 12 34 38 13  2  0  0  0]
[ 1  2 23 32 30 11  0  0  0  0]
[ 0  0  0  0  0  0  0  0  1  0]
[ 0  0  0  0  0  0  0  0  2  8]
[ 0  0  0  0  0  0  0 11 36 18]
[ 0  0  0  0  0  0  0 10 50 17]
[ 0  1  3  2  0  0  0  4 21  8]
[ 2 19 35 15  5  0  0  0  5  1]
[ 5 29 40 25  3  0  0  0  0  0]
[ 1  3  7  5  0  0  0  0  0  0]
```

- Calculate the entropy of the point distribution over cells, H:

$$H = -\sum_{i=1}^{m} p_i \log_2(p_i) \tag{5}$$

where $p_i = c_i/n$ with c_i is the number of data points in the ith cell, and n is the total number of data points in all cells.

- With $m = 100$ cells, the maximum entropy value is $\log_2 m = \log_2 100 = 6.6439$. The entropy can be normalized to $[0, 1]$ by using $H / \log_2 m$.

Validating cluster tendency with cell–based entropy (cont'd)

- Case (a):
  - Entropy = 6.5539
  - Normalized entropy = **0.9864** ≈ 1.0
- Case (b):
  - Entropy = 5.5318
  - Normalized entropy = **0.8326**
- Case (c):
  - Entropy = 4.8118
  - Normalized entropy = **0.7242**
- The smaller entropy value, the more clustered the data is. Entropy = 0 when all data points fall into one cell.
- This method also depends on the way we divide the data space into cells, i.e, the total number of cells.

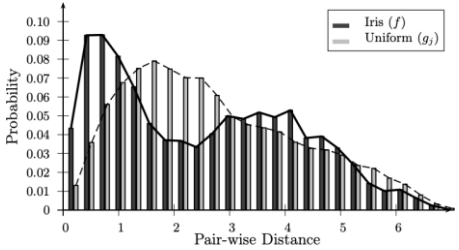
Validating cluster tendency with distance distribution

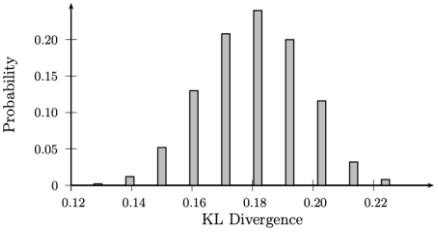
- Instead of trying to estimate the density, another approach to determine clusterability is to compare the pair–wise point distances from D, with those from the randomly generated samples R_i from the null distribution (i.e., uniformly distributed data).
- First, compute the pair–wise distance values for every pair of points in D to form a proximity matrix $\mathrm{W} = \{w_{pq}\}_{p,q=1..n}$ using some distance measure.
- Then create the EPMF from the proximity matrix W by binning the distances into b bins:

$$f(i) = P(w_{pq} \in \text{bin } i \mid \mathrm{x}_p, \mathrm{x}_q \in \mathrm{D}, p > q) = \frac{|\{w_{pq} \in \text{bin } i\}|}{n(n-1)/2} \quad (6)$$

- Likewise, for each of the (uniformly distributed) samples R_j $(j = 1..t)$, we can determine the EPMF for the pair–wise distances, denoted g_j.
- Finally, compute the KL divergences between f and g_j (for $j = 1..t$). And compute the expectation and the variance of the KL divergence values.

Example of distance distribution [4]





- Number of bins $b = 25$; $t = 500$ samples.
- KL divergence computed using logarithm with base 2. The mean divergence is $\mu_{\mathrm{KL}} = 0.18$, with standard deviation $\sigma_{\mathrm{KL}} = 0.017$.
- Even though the Iris dataset has a good clustering tendency, the KL divergence is not very large. We conclude that, at least for the Iris dataset, the distance distribution is not as discriminative as the spatial histogram approach for clusterability analysis.

Validating cluster tendency with Hopkins statistic

- Let $D = \{x_1, x_2, \ldots, x_n\}$ be a set of n data instances in R^m.
- Randomly choose h ($< n$) data instances $\{x^1, x^2, \ldots, x^h\}$ from D. For each data instance x^i, finding the distance to its closest instance in D.

$$a_i = \min_{\mathbf{u} \in \mathcal{D}} d(\mathbf{x}^i, \mathbf{u}) \tag{7}$$

- Randomly generate h pseudo data instances $\{y^1, y^2, \ldots, y^h\}$ in R^m according to uniform distribution in all m dimensions and the value range for each dimension is the same as data in D. For each data instance y^i, finding the distance to its closest instance in D.

$$b_i = \min_{\mathbf{v} \in \mathcal{D}} d(\mathbf{y}^i, \mathbf{v}) \tag{8}$$

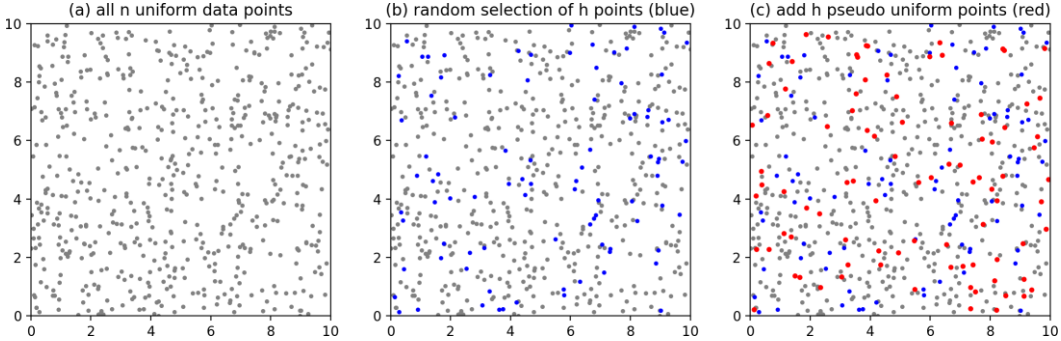
Validating cluster tendency with Hopkins statistic (cont'd)

- Hopkins statistic, H, is computed as:

$$H = \frac{\sum_{i=1}^{h} b_i}{\sum_{i=1}^{h} b_i + \sum_{i=1}^{h} a_i} \tag{9}$$

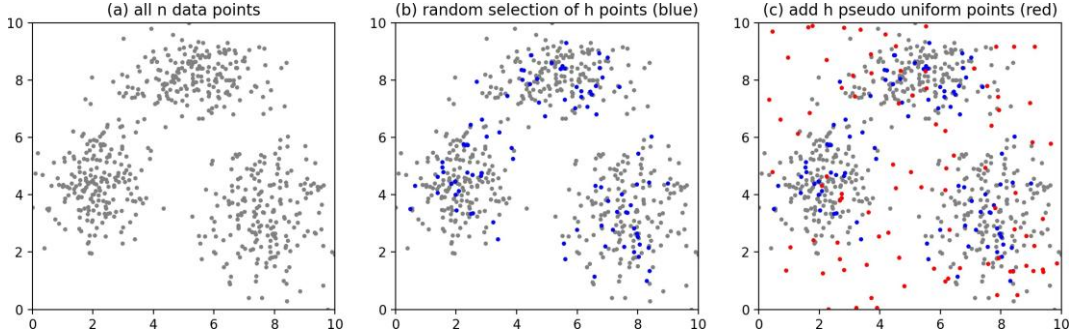
- If data in D is uniformly or near–uniformly distributed, H will be near 0.5.
- If H is close to 1.0, D has cluster structures, i.e., far from the uniform distribution.

Example of Hopkins statistic with uniformly distributed data



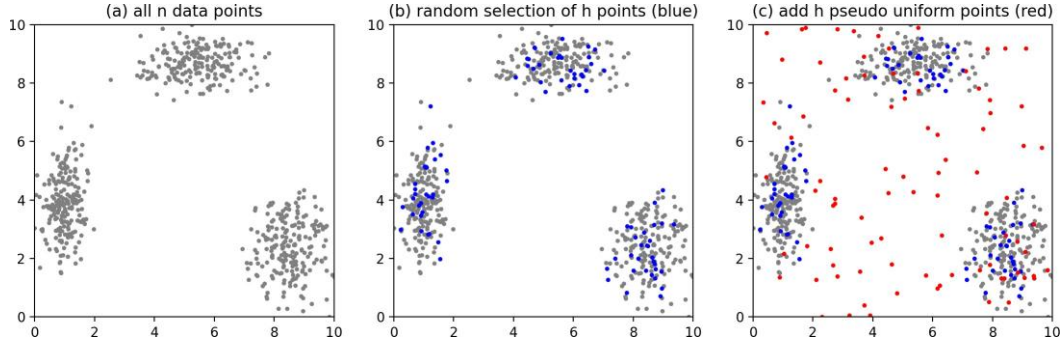
- D consists of n = 600 uniformly distributed data points, h = 90.
- $\sum_{i=1}^{h} a_i$ = 18.4981 and $\sum_{i=1}^{h} b_i$ = 19.9432.
- Hopkins statistic: H = 19.9432/(19.9432 + 18.4981) = 0.5188 ≈ **0.5**

Example of Hopkins statistic with normal distribution clusters



- D consists of $n = 600$ uniformly distributed data points, $h = 90$.
- $\sum_{i=1}^{h} a_i = 13.2464$ and $\sum_{i=1}^{h} b_i = 45.1340$.
- Hopkins statistic: $H = 45.1340/(45.1340 + 13.2464) = 0.7731$

Example of Hopkins statistic with normal distribution data (cont'd)



- D consists of $n = 600$ uniformly distributed data points, $h = 90$.
- $\sum_{i=1}^{h} a_i = 9.3838$ and $\sum_{i=1}^{h} b_i = 81.5614$.
- Hopkins statistic: $H = 81.5614/(81.5614 + 9.3838) = 0.8968$

Outline

1 Data clustering concepts

2 Data understanding before clustering

3 Hierarchical clustering

4 Partitioning clustering

5 Distribution–based clustering

6 Density–based clustering

7 Clustering validation and evaluation

8 References and Summary

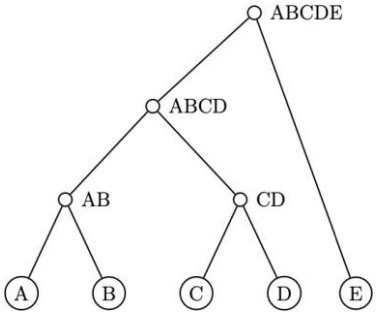
Hierarchical clustering

- Given dataset $\mathtt{D}$ consisting of n data points in a d–dimensional space, the goal of **hierarchical clustering** is to create a sequence of nested partitions, which can be conveniently visualized via a tree or hierarchy of clusters, also called the cluster **dendrogram**.
- The clusters in the hierarchy range from the **fine–grained** to the **coarse–grained**: the lowest level of the tree (*the leaves*) consists of each point in its own cluster, whereas the highest level (*the root*) consists of all points in one cluster.
- At some intermediate level, we may find meaningful clusters. If the user supplies k, the desired number of clusters, we can choose the level at which there are k clusters.
- There are two main algorithmic approaches to mine hierarchical clusters: **agglomerative** (bottom–up) and **divisive** (top–down).

Hierarchical clustering (cont'd)

- Given $D = \{x_1, x_2, \ldots, x_n\}$, where $x_i \in \mathbb{R}^d$, a clustering $C = \{C_1, C2, \ldots, C_k\}$ is a partition of D, i.e., each cluster is a set of data points $C_i \subseteq D$, such that the clusters are pairwise disjoint $C_i \cap C_j = \emptyset$ (for all $i \neq j$), and $\cup C_i = D$.
- A clustering $A = \{A_1, A_2, \ldots, A_r\}$ is said to be nested in another clustering $B = \{B_1, B_2, \ldots, B_s\}$ if and only if $r > s$, and for each cluster $A_i \in A$, there exists a cluster $B_j \in B$, such that $A_i \subseteq B_j$.
- Hierarchical clustering yields a sequence of m nested partitions $C_1, C_2, \ldots, C_m$, ranging from the trivial clustering $C_1 = \{\{x_1\}, \{x_2\}, \ldots, \{x_n\}\}$ where each point is in a separate cluster, to the other trivial clustering $C_m = \{\{x_1, x_2, \ldots, x_n\}\}$, where all points are in one cluster.
- In general, the clustering C_{t-1} is nested in the clustering C_t.
- The cluster **dendrogram** is a rooted binary tree that captures this nesting structure, with edges between cluster $C_i \in C_{t-1}$ and cluster $C_j \in C_t$ if C_i is nested in C_j, i.e., if $C_i \subset C_j$.

The dendrogram and nested clustering solutions [4]



clustering	clusters
$\mathcal{C}_1$	$\{A\}, \{B\}, \{C\}, \{D\}, \{E\}$
$\mathcal{C}_2$	$\{AB\}, \{C\}, \{D\}, \{E\}$
$\mathcal{C}_3$	$\{AB\}, \{CD\}, \{E\}$
$\mathcal{C}_4$	$\{ABCD\}, \{E\}$
$\mathcal{C}_5$	$\{ABCDE\}$

- The left figure is the dendrogram.
- The right table is the five levels of nested clustering solutions, corresponding to the dendrogram on the left.

Agglomerative hierarchical clustering

- In agglomerative hierarchical clustering, we begin with each of the n data points in a separate cluster.
- We repeatedly merge the two closest clusters until all points are members of the same cluster, as shown in the pseudo code (next slide).
- Given a set of clusters $C = \{C_1, C_2, \ldots, C_m\}$, we find the closest pair of clusters C_i and C_j and merge them into a new cluster $C_{ij} = C_i \cup C_j$.
- Next, we update the set of clusters by removing C_i and C_j and add C_{ij}, as follows $C = C \setminus \{\{C_i\} \cup \{C_j\}\} \cup \{C_{ij}\}$.
- This process is repeated until C contains only one cluster. If specified, we can stop the merging process when there are exactly k clusters remaining.

Agglomerative hierarchical clustering: the pseudo code [4]

AgglomerativeClustering$(\mathbf{D}, k)$:

1 $\mathcal{C} \leftarrow \{C_i = \{\mathbf{x}_i\} \mid \mathbf{x}_i \in \mathbf{D}\}$ `// Each point in separate cluster`
2 $\mathbf{\Delta} \leftarrow \{\delta(\mathbf{x}_i, \mathbf{x}_j) \colon \mathbf{x}_i, \mathbf{x}_j \in \mathbf{D}\}$ `// Compute distance matrix`
3 **repeat**
4 Find the closest pair of clusters $C_i, C_j \in \mathcal{C}$
5 $C_{ij} \leftarrow C_i \cup C_j$ `// Merge the clusters`
6 $\mathcal{C} \leftarrow \mathcal{C} \setminus \{\{C_i\} \cup \{C_j\}\} \cup \{C_{ij}\}$ `// Update the clustering`
7 Update distance matrix $\mathbf{\Delta}$ to reflect new clustering
8 **until** $|\mathcal{C}| = k$

Distance between clusters: different ways to merge clusters

- The main step in the algorithm is to determine the closest pair of clusters.
- The cluster–cluster distances are ultimately based on the distance between two points, which is typically computed using the Euclidean distance or L_2–norm, defined as

$$\delta(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^{d} (x_i - y_i)^2} \tag{10}$$

- There are several ways to measure the proximity between two clusters: **single link**, **complete link**, **average link**, **centroid link**, **radius**, and **diameter**.

Distance between clusters: different ways to merge clusters (cont'd)

- **Single link**:

  Given two clusters C_i and C_j , the distance between them, denoted $\delta(C_i, C_j)$ is defined as the **minimum** distance between a point in C_i and a point in C_j :

  $$\delta(C_i, C_j) = \min\{\delta(x, y) \mid x \in C_i, y \in C_j\} \quad (11)$$

  Merging any two clusters having the smallest single link distance at each iteration.

- **Complete link**:

  The distance between two clusters is defined as the **maximum** distance between a point in C_i and a point in C_j :

  $$\delta(C_i, C_j) = \max\{\delta(x, y) \mid x \in C_i, y \in C_j\} \quad (12)$$

  Merging any two clusters having the smallest complete link distance at each iteration.

Distance between clusters: different ways to merge clusters (cont'd)

- **Average link**:

  The distance between two clusters is defined as the average pairwise distance between points in C_i and points in C_j:

$$\delta(C_i, C_j) = \frac{\sum_{\mathbf{x} \in C_i} \sum_{\mathbf{y} \in C_j} \delta(\mathbf{x}, \mathbf{y})}{n_i \cdot n_j} \tag{13}$$

  where $n_i = |C_i|$ and $n_j = |C_j|$. Merging any two clusters having the smallest average link distance at each iteration.

- **Centroid link** or **mean distance**:

  The distance between two clusters is defined as the distance between the means or centroids of the two clusters:

$$\delta(C_i, C_j) = \delta(\boldsymbol{\mu}_i, \boldsymbol{\mu}_j) \tag{14}$$

  where $\boldsymbol{\mu}_i = (\sum_{\mathbf{x} \in C_i} \mathbf{x})/n_i$ and $\boldsymbol{\mu}_j = (\sum_{\mathbf{y} \in C_j} \mathbf{y})/n_i$. Merging any two clusters having the smallest centroid link distance at each iteration.

Distance between clusters: different ways to merge clusters (cont'd)

- **Radius**:

  Radius of a cluster is the distance from its centroid (mean) $\boldsymbol{\mu}$ to the furthest point in the cluster:

  $$r(C) = \max\{\delta(\boldsymbol{\mu}_C, \mathrm{x}) \mid \mathrm{x} \in C)\} \tag{15}$$

  Merging any two clusters that form a new cluster (if being merged) having smallest radius at each iteration.

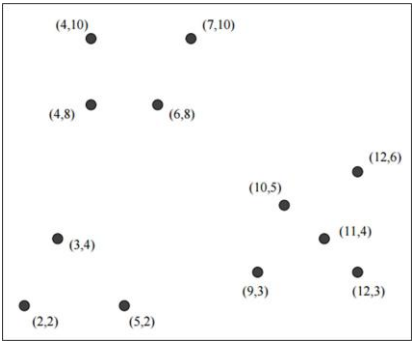
- **Diameter**:

  Diameter of a cluster is the distance between two furthest points in the cluster:

  $$d(C) = \max\{\delta(\mathrm{x}, \mathrm{y}) \mid \mathrm{x}, \mathrm{y} \in C)\} \tag{16}$$

  Merging any two clusters that form a new cluster (if being merged) having smallest diameter at each iteration.

Example of agglomerative hierarchical clustering

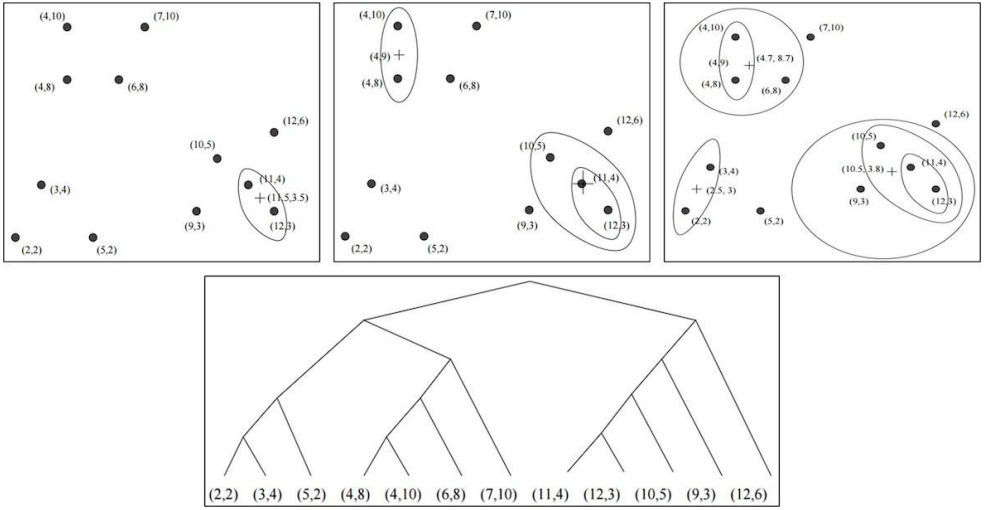


- The dataset D consists of 12 data points in R^2.
- Initially, each point is a separate cluster.
- Closest pairs of points: $\delta((10, 5), (11, 4)) = \delta((11, 4), (12, 3)) = \sqrt{2}$.

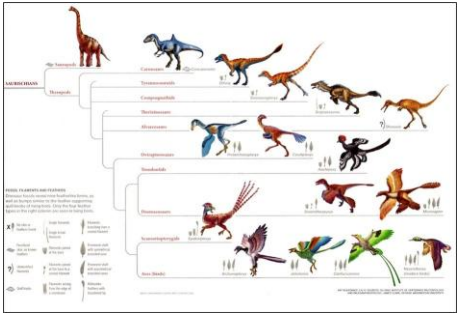
Example of agglomerative hierarchical clustering: cluster merging

[(2, 2)]	[(2, 2)]	[(2, 2)]	[(2, 2)]	[(2.5, 3)]	[(2.5, 3)]	[(2.5, 3)]
[(3, 4)]	[(3, 4)]	[(3, 4)]	[(3, 4)]			
[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]	[(5, 2)]
[(4, 8)]	[(4, 8)]	[(4, 9)]	[(4, 9)]	[(4, 9)]	[(4.7, 8.7)]	[(4.7, 8.7)]
[(4, 10)]	[(4, 10)]					
[(6, 8)]	[(6, 8)]	[(6, 8)]	[(6, 8)]	[(6, 8)]		
[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]	[(7, 10)]
[(11, 4)]	[(11.5, 3.5)]	[(11.5, 3.5)]	[(11, 4)]	[(11, 4)]	[(11, 4)]	[(10.5, 3.8)]
[(12, 3)]						
[(10, 5)]	[(10, 5)]	[(10, 5)]				
[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	[(9, 3)]	
[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]	[(12, 6)]

Example of agglomerative hierarchical clustering: the results



When should we stop merging?



- When we have a prior knowledge about the number of potential clusters in the data.
- When the merging starts produce *low–quality* clusters (e.g., the average distance from points in a cluster to its mean is larger than a given threshold).
- When the algorithm produces the whole dendrogram, e.g., an evolutionary tree.

Agglomerative clustering: computational complexity

- Compute the distance of each cluster to all other clusters, and at each step the number of clusters decreases by one. Initially it takes $O(n^2)$ time to create the pairwise distance matrix, unless it is specified as an input to the algorithm.
- At each merge step, the distances from the merged cluster to the other clusters have to be recomputed, whereas the distances between the other clusters remain the same. This means that in step t, we compute $O(n - t)$ distances.
- The other main operation is to find the closest pair in the distance matrix. For this we can keep the n^2 distances in a heap data structure, which allows us to find the minimum distance in $O(1)$ time; creating the heap takes $O(n^2)$ time.
- Deleting/updating distances from the merged cluster takes $O(\log n)$ time for each operation, for a total time across all merge steps of $O(n^2 \log n)$.
- Thus, the computational complexity of hierarchical clustering is $O(n^2 \log n)$.

Outline

1. Data clustering concepts
2. Data understanding before clustering
3. Hierarchical clustering
4. Partitioning clustering
5. Distribution–based clustering
6. Density–based clustering
7. Clustering validation and evaluation
8. References and Summary

Partitioning clustering methods

- The simplest and most fundamental version of cluster analysis is partitioning, which organizes the objects of a set into several exclusive groups or clusters.
- We can assume that the number of clusters is given as background knowledge. This parameter is the starting point for partitioning methods.
- Formally, given a data set, D, of n objects, and k, the number of clusters to form, a **partitioning algorithm** organizes the objects into k partitions ($k \leq n$), where each partition represents a cluster.
- The clusters are formed to optimize an **objective partitioning criterion**, such as a dissimilarity function based on distance, so that the objects within a cluster are "similar" to one another and "dissimilar" to objects in other clusters in terms of the data set attributes.
- Most popular partitioning algorithms are k–means, k–mediods, and k–medians. These methods use a centroid point to represent each cluster. Thus, they are also called **representative** or **centroid** methods.

Data clustering problem revisited



- Let $X = (X_1, X_2, \ldots, X_d)$ be a d–dimensional space, where each attribute/variable X_j is numeric or categorical.
- Let $D = \{x_1, x_2, \ldots, x_n\}$ be a data sample or dataset consisting of n data points (a.k.a data instances, observations, examples, or tuples) $x_i = (x_1, x_2, \ldots, x_d) \in X$.
- Data clustering is to use a clustering technique or algorithm A to assign data points in D into their most likely clusters. The clustering results are a set of k clusters $C = \{C_1, C_2, \ldots, C_k\}$. Data points in the same cluster are similar to each other in some sense and far from the data points in other clusters.

- Let $\mathtt{C} = \{C_1, C_2, \ldots, C_k\}$ be a clustering solution, we need some scoring function that evaluates its quality or goodness on $\mathtt{D}$. This *sum of squared errors* scoring function is defined as:

$$SSE(\mathcal{C}, \mathcal{D}) = \sum_{i=1}^{k} \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2 \tag{17}$$

- The goal is to find the clustering solution $\mathtt{C}^*$ that minimizes the SSE score:

$$\begin{aligned} \mathcal{C}^* &= \arg\min_{\mathcal{C}} SSE(\mathcal{C}, \mathcal{D}) \\ &= \arg\min_{\mathcal{C}} \left(\sum_{i=1}^{k} \sum_{\mathbf{x}_j \in C_i} \|\mathbf{x}_j - \boldsymbol{\mu}_i\|^2 \right) \end{aligned} \tag{18}$$

- *K*–means algorithm employs a **greedy iterative approach** to find a clustering solution that minimizes the SSE objective. As such it can converge to a local optima instead of the globally optimum clustering.

K–means algorithm (cont'd)

- K–means initializes the cluster means by randomly generating k points in the data space. This is typically done by generating a value uniformly at random within the range for each dimension.
- Each iteration of k–means consists of two steps:
  - Cluster assignment, and
  - Centroid or mean update.
- Given the k cluster means, in the **cluster assignment step**, each point $\mathrm{x}_j \in \mathrm{D}$ is assigned to the **closest mean**, which induces a clustering, with each cluster C_i comprising points that are closer to $\boldsymbol{\mu}_i$ than any other cluster mean. That is, each point x_j is assigned to cluster C_{j^*}, where

$$J^* = \arg\min_{i=1..k} \|\mathrm{x}_j - \boldsymbol{\mu}_i\|^2 \tag{19}$$

K–means algorithm (cont'd)

- Given a set of clusters C_i, $i = 1..k$, in the **centroid update step**, new mean values are computed for each cluster from the points in C_i.
- The cluster assignment and centroid update steps are carried out iteratively until we reach a fixed point or local minima.
- Practically speaking, one can assume that k–means has converged if the centroids do not change from one iteration to the next. For instance, we can stop if

$$\sum_{i=1}^{k} \|\boldsymbol{\mu}_i^t - \boldsymbol{\mu}_i^{t-1}\|^2 \leq \epsilon \tag{20}$$

where $\varepsilon > 0$ is the convergence threshold, and t denotes the current iteration.

K–means algorithm: the pseudo code [4]

K-MEANS $(\mathbf{D}, k, \epsilon)$:

$t = 0$

Randomly initialize k centroids: $\boldsymbol{\mu}_1^t, \boldsymbol{\mu}_2^t, \ldots, \boldsymbol{\mu}_k^t \in \mathbb{R}^d$

repeat

  $t \leftarrow t + 1$

  `// Cluster Assignment Step`

  foreach $\mathbf{x}_j \in \mathbf{D}$ **do**

    $j^* \leftarrow \arg\min_i \left\{ \left\| \mathbf{x}_j - \boldsymbol{\mu}_i^t \right\|^2 \right\}$ `// Assign` $\mathbf{x}_j$ `to closest centroid`

    $C_{j^*} \leftarrow C_{j^*} \cup \{\mathbf{x}_j\}$

  `// Centroid Update Step`

  foreach $i = 1$ *to* k **do**

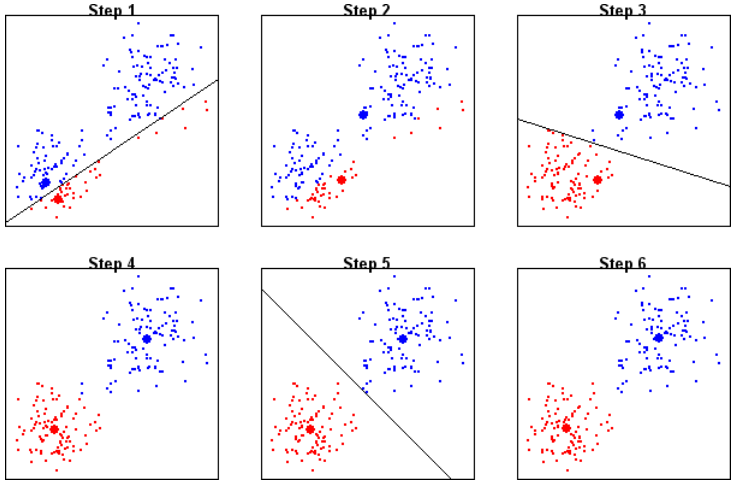
    $\boldsymbol{\mu}_i^t \leftarrow \frac{1}{|C_i|} \sum_{\mathbf{x}_j \in C_i} \mathbf{x}_j$

until $\sum_{i=1}^{k} \left\| \boldsymbol{\mu}_i^t - \boldsymbol{\mu}_i^{t-1} \right\| \leq \epsilon$

K–means algorithm: computational complexity

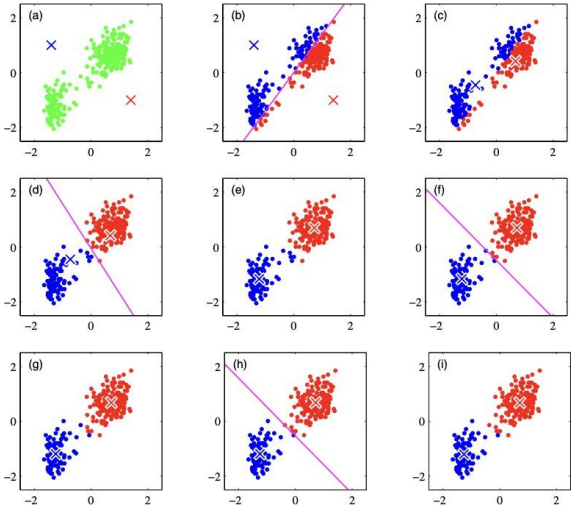
- The cluster assignment step take $O(nkd)$ time, since for each of the n points we have to compute its distance to each of the k clusters, which takes d operations in d dimensions.
- The centroid re–computation step takes $O(nd)$ time, since we have to add at total of n d–dimensional points.
- Assuming that there are t iterations, the total time for k–means is $O(tnkd)$.
- In terms of the I/O cost it requires $O(t)$ full database scans, since we have to read the entire database in each iteration.

K–means algorithm: example 1



Clustering with k–means [source: sherrytowers.com/2013/10/24/k-means-clustering]

K–means algorithm: example 2



Clustering with k–means [from *Pattern Recognition and Machine Learning* by C.M. Bishop]

K–means algorithm: example 3 (image segmentation)



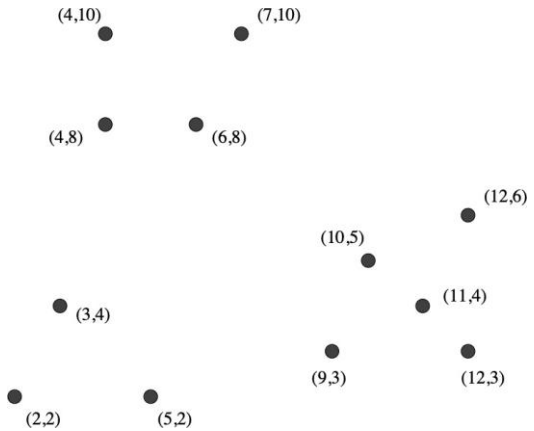
Image segmentation with k–means [from *Pattern Recognition and Machine Learning* by C.M. Bishop]

Initialization for k mean vectors $\boldsymbol{\mu}_i$

- The initial means should lay in different clusters. There are two approaches:
  - Pick points that are as far away from one another as possible.
  - Cluster a (small) sample of the data, perhaps hierarchically, so there are k clusters. Pick a point from each cluster, perhaps that point closest to the centroid of the cluster.
- The second approach requires little elaboration.
- For the first approach, there are several ways. One good choice is:

```
Pick the first point at random;
WHILE there are fewer than k points DO
    Add the point whose minimum distance from the selected points is as large as possible;
END
```

Initialization for k mean vectors $\boldsymbol{\mu}_i$: example



Initial selection for mean values [from *Mining of Massive Datasets* by J. Leskovec et al.]

Initialization for k mean vectors $\boldsymbol{\mu}_i$: example (cont'd)

- In the worst case, our initial choice of a point is near the center, say $(6, 8)$. The furthest point from $(6, 8)$ is $(12, 3)$, so that point is chosen next.
- Among the remaining ten points, the one whose minimum distance to either $(6, 8)$ or $(12, 3)$ is a maximum is $(2, 2)$. That point has distance $\sqrt{52} = 7.21$ from $(6, 8)$ and distance $\sqrt{101} = 10.05$ to $(12, 3)$; thus its "score" is 7.21. We can check easily that any other point is less than distance 7.21 from at least one of $(6, 8)$ and $(12, 3)$. Notice that these three belong to different clusters.
- Had we started with a different point, say $(10, 5)$, we would get a different set of three initial points. In this case, the starting points would be $(10, 5)$, $(2, 2)$, and $(4, 10)$. Again, these points belong to the three different clusters.

K–means is sensitive to outliers

- The k–means algorithm is **sensitive to outliers** because such objects are far away from the majority of the data, and thus, when assigned to a cluster, they can dramatically distort the mean value of the cluster. This inadvertently affects the assignment of other objects to clusters. This effect is more serious due to the use of the *squared error*.
- Example: consider 7 data points in the 1–d space: 1, 2, 3, 8, 9, 10, 25, with $k = 2$.
  - Intuitively, by visual inspection we may imagine the points partitioned into the clusters {1, 2, 3} and {8, 9, 10}, where point 25 is excluded because it appears to be an outlier. How would k–means partition the values with $k = 2$?
  - Solution1: {1, 2, 3} with mean = 2 and {8, 9, 10, 25} with mean = 13. The error is:

$$(1-2)^2 + (2-2)^2 + (3-2)^2 + \cdots + (10-13)^2 + (25-13)^2 = 196$$

  - Solution2: {1, 2, 3, 8} with mean = 3.5 and {9, 10, 25} with mean = 14.67. The error is:

$$(1-3.5)^2 + (2-3.5)^2 + (3-3.5)^2 + (10-14.67)^2 + (25-14.67)^2 = 189.67.$$

  - The Solution2 is chosen because it has the lowest squared error. However, 8 should not be in cluster 1. In addition, the mean of the second cluster is 14.67 and quite far from 9 and 10 due to the outlier 25.

K–medoids clustering algorithm

- Rather using mean values, k–mediods pick actual data objects in the dataset to represent the clusters, using one representative object per cluster.
- Each remaining object is assigned to the cluster of which the representative object is the most similar.
- The partitioning method is then performed based on the principle of minimizing the sum of the dissimilarities between each object x and its corresponding representative object o_i. That is, an **absolute–error criterion** is used, defined as:

$$E = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{o}_i\|_1 \qquad (21)$$

  This is the basis for the k–medoids method, which groups n objects into k clusters by minimizing the absolute error.

- When $k = 1$, we can find the exact median in $O(n^2)$ time. However, when k is a general positive number, the k–medoid problem is NP-hard.

K–mediods: partitioning around mediods (PAM) algorithm

- The **partitioning around medoids** (**PAM**) algorithm is a popular realization of k–medoids clustering. It tackles the problem in an **iterative**, **greedy** way.
- Like the k–means algorithm, the initial representative objects (called seeds) are chosen arbitrarily.
- We consider whether replacing a representative object by a non–representative object would improve the clustering quality. All the possible replacements are tried out.
- The iterative process of replacing representative objects by other objects continues until the quality of the resulting clustering cannot be improved by any replacement.
- This quality is measured by a cost function of the sum of dissimilarity between every data object and the representative object of its cluster (equation 21).

K–mediods: partitioning around mediods (PAM) algorithm (cont'd)

- Specifically, let $o_1, o_2, \ldots, o_k$ be the current set of representative objects (i.e., medoids) of the k clusters.
- To determine whether a non–representative object, denoted by o_{random}, is a good replacement for a current medoid o_j ($1 \leq j \leq k$), we calculate the distance from every object x to the closest object in the set $\{o_1, \ldots, o_{j-1}, o_{random}, o_{j+1}, \ldots, o_k\}$, and use the distance to update the cost function.
- The reassignments of objects to $\{o_1, \ldots, o_{j-1}, o_{random}, o_{j+1}, \ldots, o_k\}$ are simple:
  - Suppose an object x is currently assigned to a cluster represented by mediod o_j: x needs to be reassigned to either o_{random} or some other cluster represented by o_i ($i \neq j$), whichever is the closest.
  - Suppose an object x is currently assigned to a cluster represented by some other o_i ($i \neq j$): x remains assigned to o_i as long as x is still closer to o_i than to o_{random}. Otherwise, x is reassigned to o_{random}.
- If the error E (equation 21) decreases, replace o_j with o_{random}. Otherwise, o_j is acceptable and nothing is changed in the iteration. The algorithm will stop when there is no change in error E with all possible replacements.

Which method is more robust? k–means or k–medoids?

- The k–medoids method is more robust than k–means in the presence of noise and outliers because a medoid is less influenced by outliers or other extreme values than a mean.
- However, the complexity of each iteration in the k–medoids algorithm is $O(k(nk)^2)$.
- For large values of n and k, such computation becomes **very costly**, and much more costly than the k–means method.
- Both methods require the user to specify k, the number of clusters.
- A typical k–medoids partitioning algorithm like PAM works effectively for small data sets, but does not scale well for large data sets. How can we scale up the k–medoids method? To deal with larger data sets, a sampling–based method called **CLARA** (**Clustering LARge Applications**) can be used.

K–medians clustering algorithm

- In the k–medians algorithm, the Manhattan distance (L_1 distance) is used in the objective function rather than the Euclidean (L_2 distance). The objective function in k–medians is:

$$E = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \|\mathbf{x} - \mathbf{m}_i\|_1 = \sum_{i=1}^{k} \sum_{\mathbf{x} \in C_i} \sum_{j=1}^{d} |x_j - m_j| \tag{22}$$

  where m_i is the median of the data points along each dimension in cluster C_i. This is because the point that has the minimum sum of L_1–distances to a set of points distributed on a line is the median of that set.

- As the median is chosen independently along each dimension, the resulting d–dimensional representative will (typically) not belong to the original dataset $\mathtt{D}$. The k–medians approach is sometimes confused with the k–medoids approach, which chooses these representatives from the original database $\mathtt{D}$.

- The k–medians approach generally selects cluster representatives in a more robust way than k–means, because the median is not as sensitive to the presence of outliers in the cluster as the mean.

Outline

1. Data clustering concepts
2. Data understanding before clustering
3. Hierarchical clustering
4. Partitioning clustering
5. Distribution–based clustering
6. Density–based clustering
7. Clustering validation and evaluation
8. References and Summary

References

1 J. Han, M. Kamber, and J. Pei. *Data Mining: Concepts and Techniques*. Morgan Kaufmann, Elsevier, 2012 [Book1].

2 C. Aggarwal. *Data Mining: The Textbook*. Springer, 2015 [Book2].

3 J. Leskovec, A. Rajaraman, and J. D. Ullman. *Mining of Massive Datasets*. Cambridge University Press, 2014 [Book3].

4 M. J. Zaki and W. M. Jr. *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2013 [Book4].

5 D. Easley and J. Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010 [Book5].

6 J. VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly, 2017 [Book6].

7 J. Grus. *Data Science from Scratch: First Principles with Python*. O'Reilly, 2015 [Book7].

Summary

- Introducing important concepts of clustering: definitions, types of clustering (hard vs. soft), main requirements for clustering, clustering approaches, challenges in clustering, and clustering applications.
- Main techniques to understanding the data distribution before clustering: spatial histogram, cell–based entropy, distance distribution, and Hopkins statistic.
- The hierarchical clustering approach with agglomerative method (bottom–up), dendrogram, different ways to merge clusters (single link, complete link, average link, centroid link, radius, and diameter).
- The partitioning approach with k–means algorithm, the initialization of k centroids, and the variants of k–means including k–mediods (PAM algorithm) and k–medians.